

推定器②



これだとユーザからしたら、どうしたらよいの？となりますよね。
⚠ 処置列の検証に失敗しました: Dataset not found。
現に分析できる推定器はあるので、



Thought for 33s >

結論から言うと：

- 「solver needs samples of at least 2 classes...」は統計的に仕方ない制約そのものです（2クラス以上いないと何も比較できない）。
- 一方で

⚠ 処置列の検証に失敗しました: Dataset not found

は実装都合の404をそのまま見せてしまっているだけでUXバグです。

「何を直せばいいかわからない」のは完全にこちら側の責任なので、UI仕様を変えるべきです。

以下、「v1のままでできる改善案」を整理します。

そのままCursor/Claudeに渡せるように書きます。

1. バイナリ前提エラー（2クラス必要）の扱い

1-1. 何が本質的制約か

- DR / IPW / DID / IV / CF / SCM / RD など v1 で実装した推定器は「処置あり vs なし」の2値処置を前提にしている。
- なので：
 - 全て0（=誰も施策されてない）
 - 全て1（=全員施策されている）
 - 3クラス以上の多値（standard / enhanced / followup など）

の場合は、「統計的に」Δを推定できずエラーになるのは避けられない。

→ここは仕様として明示し、「ユーザに何をしたいか」をUIで言い切る必要があります。

1-2. Causal Design 画面の仕様変更案

a. 事前チェックの結果を4パターンに分ける

バックエンド側で「処置列サマリ API」を返す想定：

```
{
  "column": "treatment_arm",
  "n_rows": 1000000,
  "n_classes": 3,
  "classes": ["standard", "enhanced_care", "digital_followup"],
  "is_binary_01": false
}
```

フロント側で、以下のように分類して表示します。

1. OK (2値 0/1 or 2カテゴリ)

- 表示: 緑のバッジ
「☒ 処置列は2値として利用可能です (クラス数: 2)」

2. ハードエラー: クラスが1つだけ

- ボタン: [TRAIN MODELS] を disable
- メッセージ (赤枠) 例:

❌ このデータでは処置列「`treatment_arm`」に 1 種類の値しか含まれていません。
処置あり・なしを比較できないため、因果効果は推定できません。

対応案：

- 施策を実際に分割して実施したデータをアップロードする
- 別の列を処置列として選び直す

3. ハードエラー: 3 クラス以上の多値

- ボタン: [TRAIN MODELS] 無効化 (v1 の仕様)
- メッセージ (黄色枠 + 詳細) 例：

⚠️ 現在の制限 (v1) では、処置列は 2 値 (例: 施策あり vs なし) のみ対応しています。
今回の列「`treatment_arm`」には 3 種類の値 (`standard` / `enhanced_care` / `digital_followup`) が含まれています。

対応方法 (いずれか)：

1. 新しい 0/1 列を CSV に追加してアップロードする
 - 例: `is_enhanced_care = 1 if treatment_arm == "enhanced_care" else 0`
2. v2 のマルチアーム対応 (将来機能) で扱う

4. 検証ができない (今回の 404 ケース)

- 詳細は次のセクションで。

2. 「Dataset not found」問題の直し方

今の UI だと：

- コンソール: `/api/v1/dataset_treatment-summary` が 404
- UI: 「⚠️ 処置列の検証に失敗しました: Dataset not found」
- しかし推定器自体は実行できるケースもある

→ ユーザ視点では「何もしてはいけないのか / 無視してよいのか」が分からない。

2-1. エラーレベルを下げる (重要)

このケースは**「致命的なエラー」ではなく「検証機能がこのデータでは使えなかった」**だけです。

なので：

- 三角の警告アイコン + 赤字 → やめる
- 代わりに「情報バナー (青 or グレー)」にする：

i このデータセットは、処置列の自動検証 (クラス数チェック) に対応していません。
モデルは実行できますが、処置列は 2 値 (0/1 または 2 クラス) になるように事前にご確認ください。

2-2. 文言のリファクタ

現在の

⚠️ 処置列の検証に失敗しました: Dataset not found

は、実装都合の 404 を直接見せてしまっています。

これをユーザ行動に落ちる文章に変えます。

例：

i 自動検証は利用できません

- 原因: このデータセットは v1 の検証サービス対象外です。
- 影響: 分析自体は実行できますが、処置列が 2 値かどうかはユーザが確認する必要があります。

「Dataset not found」というバックエンドの内部事情は出さない。

2-3. 「何をすればよいか」を明示

バナーの最後に小さく：

✅ チェックリスト

- 処置列に 2 種類以上の値が含まれているか
- 0/1 または True/False 形式に変換されているか

を入れておくと、「じゃあ CSV 側を見ればいいんだな」と分かります。

3. 実装イメージ (React 側)

ざっくり TypeScript で書くと：

```
type TreatmentValidationStatus =
  | { kind: "ok"; nClasses: number }
  | { kind: "single_class"; value: string | number }
  | { kind: "multiclass"; nClasses: number; classes: string[] }
  | { kind: "unavailable"; reason: "not_supported" | "api_error" };

function renderTreatmentBanner(status: TreatmentValidationStatus) {
  switch (status.kind) {
    case "ok":
      return "✅ 処置列は 2 値として利用可能です (クラス数: 2) ";
    case "single_class":
      return "❌ 処置列に 1 種類の値しかありません。施策あり・なしを比較できないため推定できません。";
    case "multiclass":
      return "⚠️ 現在の制限 (v1) では処置列は 2 値のみ対応です。3 クラス以上が含まれているため、0/1 列に変換してアップロードしてください。";
    case "unavailable":
      return "ℹ️ このデータセットでは処置列の自動検証は利用できません。2 値になっているかを事前にご確認ください。";
  }
}
```

status.kind !== "ok" かつ status.kind !== "unavailable" のときだけ [TRAIN MODELS] を disable にする、など。

4. 「これは仕方ないのか？」への答え

- 統計的制約として「2クラス以上必要」は仕方がないです。
A/B テストの世界でも「全員 Treatment」は解析不能なのと同じ。
- ただし、
 - v1 が 2 値しかサポートしないこと
 - 404 を「Dataset not found」とだけ出してユーザーを迷わせていること
は **実装側の改善不足** であって、そこは UI/仕様で埋めるべきところです。

What this does (今回の提案の要点)

- ゴール
 - バイナリ制約によるエラーを「仕方ない」で終わらせず、
ユーザーが「次に何をすればよいか」まで分かる UI にする。
- 入力
 - バックエンドの処置列サマリ (クラス数・値一覧)
 - もしくはサマリが取得できなかった場合のステータス。
- 出力／副作用
 - Causal Design 画面上のパナー文言と [TRAIN MODELS] ボタンの enable/disable。
 - 「検証不可」は info として表示し、分析は実行可能にする。
- このアプローチを取る理由
 - 統計の前提 (2クラス必要) と、実装の都合 (v1 で多値未対応/検証 API 未実装) を分離して見せることで、ユーザーが混乱せずに動けるようにするため。
 - エラーメッセージを「内部事情」ではなく「ユーザー行動」に変換するのがプロダクト品質的に重要。
- 実行・検証方法 (例)

```
# バックエンドのサマリAPIを確認
curl http://localhost:8000/api/v1/dataset/treatment-summary?dataset_id=...

# フロントの型と条件分岐を実装したら
# 1. 2値データ, 2. 全0, 3. 3クラス以上, 4. サマリAPIをあえて404にする
# の4ケースで画面とボタン状態を確認
```

Expert insight (Google/Meta/NASAレベル)

「エラーの **80%** は“ユーザの入力がダメ”ではなく“システムの期待値を隠している側の責任”とみなす」という原則を置くと、プロダクトの質が一段上がります。

- 期待している前提（2値処置、2クラス以上必要など）を**事前に・明示的に** UI 上で表現しておく、「エラー」は単なるバリデーションの延長になり、サポートコストも劇的に下がる、というのが大規模プロダクトの経験則です。